BUILD SPEC · INTERNAL EDITION · V2.0

LionsList

A used-goods marketplace for verified Columbia students only. You sign in with your **columbia.edu** address, you see what other students are selling, and you filter the feed by what you have in common with them — same college, same country, same neighborhood, same year.

Scope · internal listings only **Stack** · Next.js + FastAPI + Postgres **Build** · 4 weeks, 4 people
Cost · ≈ $33/month

---

01 Summary

What we are building

One website. A student signs in with their Columbia email, fills in a short profile, and lands on a feed of items other Columbia students are selling. They can narrow that feed to sellers who share their college, nationality, location, or year. When they find something, they tap *Contact seller* and get the seller's email and phone number to arrange the handoff themselves.

Trust comes from the fact that everyone on the platform is a verified Columbia student — not from star ratings, not from payments held in escrow. That is the whole bet, and it is small enough to build properly.

What changed from v1

REMOVED

- **The external tier.** No eBay Browse API, no scraped or hand-entered outside listings, no `source` column, no link expiry, no seed scripts.
- **The industry field.** Most students don't have one; it was always the weakest signal.
- **Everything that existed to serve those two** — the outbound-click event, the admin import path, the internal-vs-external comparison.

KEPT AND SIMPLIFIED

- **One table of listings**, all from real students. No tier logic anywhere.
- **Seven profile fields, six item fields.** Every profile field is a dropdown, so matching is exact.
- **Overlap-only disclosure.** You learn a seller's nationality only if it is also yours.
- **Match badges and filters**, which are still the thing worth measuring.

THE ONE HONEST CAVEAT

Dropping the external tier means the site launches empty. Nothing in the software fixes that — you need roughly **20 real listings on day one** from students you recruit by hand before you open sign-ups. Budget a week for it and treat it as a build task, not an afterthought.

Two tables carry the product

Users and listings. Photos hang off listings; events hang off everything and exist only so the analysis in section 4 is possible. Nothing else.

User

FIELD	TYPE	HOW IT IS SET	WHO CAN SEE IT
`email`	text	Verified at sign-in, never editable	Only after Contact seller
`user_name`	text	Typed once at onboarding	Everyone
`phone_number`	text, optional	Typed at onboarding	Only after Contact seller
`nationality`	2-letter country code	Dropdown	Only students who share it
`college`	enum	Dropdown, prefilled from email subdomain	Only students who share it
`grade`	enum	Dropdown	Only students who share it
`location`	enum	Dropdown of campus neighborhoods	Only students who share it

Every one of the four matching fields is a fixed dropdown, never free text — that is what makes `=` a valid comparison and keeps the filters honest. `college` covers CC, SEAS, GS, CBS, SIPA, TC, Law, GSAS, Journalism, Mailman, SPS, GSAPP, other. `grade` covers freshman through senior, masters, PhD, alumni.

Item

FIELD	TYPE	NOTES
`title`	text	Required, ≤ 80 characters
`description`	text	Optional, free text
`category`	enum	furniture · appliances · electronics · textbooks · clothing · kitchen · decor · sports · tickets · other
`price`	integer (cents)	0 allowed — free giveaways are real
`condition`	enum	`used` or `unused`
`photos`	1–5 images	At least one required to publish

SCHEMA

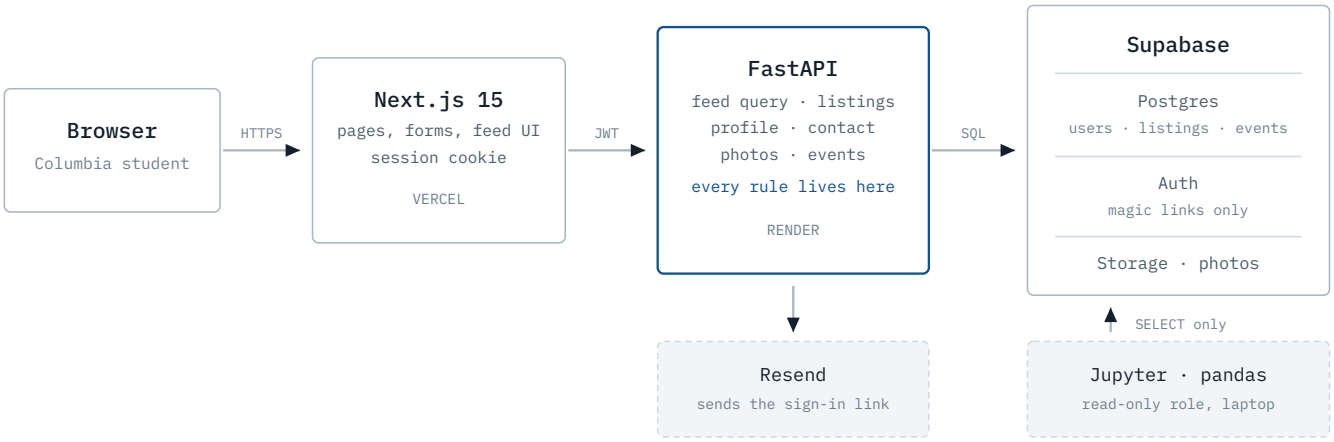
```
-- fixed vocabularies: change one = one migration line
CREATE TYPE college   AS ENUM ('cc','seas','gs','cbs','sipa','tc','law',
  'gsas','journalism','mailman','sps','gsapp','other');
CREATE TYPE grade     AS ENUM ('freshman','sophomore','junior','senior',
  'masters','phd','alumni');
CREATE TYPE location  AS ENUM ('morningside','manhattanville','uws','ues',
  'harlem','washington_heights','downtown','brooklyn','queens','other');
CREATE TYPE category  AS ENUM ('furniture','appliances','electronics','textbooks',
  'clothing','kitchen','decor','sports','tickets','other');
CREATE TYPE item_condition AS ENUM ('used','unused');
CREATE TYPE listing_status AS ENUM ('active','sold','delisted');
CREATE TYPE event_type AS ENUM ('session_start','feed_view','filter_toggle',
  'listing_view','contact_click','listing_posted','listing_edited','listing_sold');
```

```sql
CREATE TABLE users (
  id            uuid PRIMARY KEY REFERENCES auth.users(id),
  email         text NOT NULL UNIQUE,        -- verified columbia.edu
  user_name     text NOT NULL,
  phone_number  text,
  nationality   char(2)  NOT NULL,            -- ISO 3166-1, e.g. 'KR'
  college       college  NOT NULL,
  grade         grade    NOT NULL,
  location      location NOT NULL,
  deleted_at    timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE listings (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  seller_id    uuid NOT NULL REFERENCES users(id),
  title        text NOT NULL,
  description  text,
  category     category NOT NULL,
  price_cents  integer  NOT NULL CHECK (price_cents >= 0),
  condition    item_condition NOT NULL,
  status       listing_status NOT NULL DEFAULT 'active',
  created_at   timestamptz NOT NULL DEFAULT now(),
  sold_at      timestamptz
);

CREATE TABLE photos (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id  uuid NOT NULL REFERENCES listings(id) ON DELETE CASCADE,
  storage_path text NOT NULL,
  sort_order  smallint NOT NULL DEFAULT 0
);

CREATE TABLE events (             -- append-only; feeds section 4
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type  event_type NOT NULL,
  actor_id    uuid REFERENCES users(id),
  session_id  uuid NOT NULL,
  listing_id  uuid REFERENCES listings(id),
  payload     jsonb NOT NULL DEFAULT '{}',
  created_at  timestamptz NOT NULL DEFAULT now()
);
```

Seller attributes are never copied onto a listing. They are joined at query time, so editing your profile immediately corrects every badge on every item you have posted. Deleting is a status change, never a row deletion — the events table still references it.

Three pieces and one database

A Next.js site the student sees, a FastAPI service that holds every rule, and a Supabase project that holds the database, the sign-in, and the photos. The browser never talks to the database directly.



Photos reach Storage only through FastAPI, so the resize-and-strip-metadata step cannot be skipped. The notebook connects with its own read-only database user and can never write.

PIECE	CHOICE	WHY
Frontend	Next.js 15 + TypeScript + Tailwind	Server-rendered pages, shareable filter URLs, deploys from GitHub
Backend	FastAPI + Uvicorn	Typed endpoints, free API docs, same language as the analysis
Database	Postgres on Supabase	Database, sign-in and file storage from one dashboard
Data access	SQLAlchemy 2 + Alembic	Python models for writes, plain SQL for the feed, versioned migrations
Sign-in	Supabase magic links + Resend	No passwords to store, no reset flow, no breach liability
Photos	Supabase Storage + Pillow	Resize to 1600px, re-encode to WebP, strip location metadata
Analysis	Jupyter + pandas + statsmodels	Runs on a laptop against a read-only role
Hosting	Vercel + Render	Push to main, both redeploy. Nobody logs into a server.

The feed is one SQL query

Filters, match badges and the overlap-only privacy rule all resolve in a single statement. Privacy is structural: an attribute you don't share never leaves the database, so no frontend bug can leak it.

```
SELECT
  l.id, l.title, l.category, l.price_cents, l.condition, l.created_at,

  -- match badges: what the viewer shares with the seller
  (s.college     = :v_college)     AS same_college,
  (s.nationality = :v_nationality) AS same_country,
  (s.location    = :v_location)    AS same_location,
```

```
  (s.grade       = :v_grade)       AS same_grade,

  -- overlap-only: a value leaves the DB only when it is shared
  CASE WHEN s.college     = :v_college     THEN s.college     END AS seller_college,
  CASE WHEN s.nationality = :v_nationality THEN s.nationality END AS seller_country,
  CASE WHEN s.location    = :v_location    THEN s.location    END AS seller_location,

  count(*) OVER ()  AS total_count   -- the live "N items" number, free

FROM listings l
JOIN users s ON s.id = l.seller_id
WHERE l.status = 'active' AND s.deleted_at IS NULL
  AND (NOT :f_college     OR s.college     = :v_college)
  AND (NOT :f_nationality OR s.nationality = :v_nationality)
  AND (NOT :f_location    OR s.location    = :v_location)
  AND (NOT :f_grade       OR s.grade       = :v_grade)
  AND (:category::category IS NULL OR l.category = :category)
ORDER BY l.created_at DESC
LIMIT 40 OFFSET :offset;
```

Twelve endpoints, and that is all

#	ENDPOINT	DOES
1	`POST /v1/auth/magic-link`	Checks the domain, asks Supabase to email a link
2	`GET /auth/callback`	Next.js route: trades the link code for a session cookie
3	`POST /v1/auth/sign-out`	Ends the session
4	`PUT /v1/profile`	Onboarding and later edits
5	`DELETE /v1/profile`	Erases personal fields, delists items, blocks sign-in
6	`GET /v1/feed`	Runs the query above; logs `feed_view`
7	`GET /v1/listings/{id}`	Item detail with badges and the shared-only seller card
8	`POST /v1/listings`	Publish; requires at least one processed photo
9	`PATCH /v1/listings/{id}`	Edit, mark sold, delist — owner only
10	`POST /v1/photos`	Validate, resize, strip metadata, store
11	`POST /v1/listings/{id}/contact`	Returns the seller's email and phone; logs `contact_click`
12	`POST /v1/events`	Records toggles and views from the browser

What Python can tell you

One notebook, a read-only database user, pandas for the shaping and statsmodels for the confidence intervals. Everything below comes from the `events` table plus the two content tables — no extra tracking, no third-party analytics vendor.

QUESTION	METHOD	WHAT YOU LEARN
Do match badges change behavior?	Show badges on a random half of feed impressions, compare contact rate. Two-proportion test.	The one genuinely causal result. Same items, same prices — only the badge differs.
How tight can a circle get before it stops working?	Median result count at each filter combination, against the chance of a contact within 7 days.	The minimum viable community size — the number that decides whether this scales past Columbia.
Which filter do people actually use?	Toggle frequency and retention per filter, divided by the inventory each one yields.	Whether college, country, neighborhood or year is the real trust signal.
Where do people give up?	Median `result_count` at the moment a filter is switched back off.	The empty-feed threshold — how few items it takes before someone bails.
What does a used dorm couch cost?	`groupby(["category","condition"])` price distributions.	Pricing guidance you can put in the posting form, and the used/unused premium.
How fast does anything sell?	Days between `created_at` and `sold_at`, by category and price band.	Which categories have real demand and which are dead weight in the feed.
Is there a moving season?	Weekly posts and sales per active seller, straddling May or August.	Whether demand spikes at move-out — normalized so platform growth doesn't fake it.
Where does the funnel leak?	Impressions → detail views → contacts, per session.	Whether the problem is the feed, the item page, or the contact step.

THE BADGE EXPERIMENT, IN FULL

```
import pandas as pd
from sqlalchemy import create_engine
from statsmodels.stats.proportion import proportions_ztest, proportion_confint

engine = create_engine(READONLY_DATABASE_URL)

# every feed impression, flat: one row per listing shown
imp = pd.read_sql("""
    SELECT e.session_id,
           (i.item->>'id')::uuid            AS listing_id,
           (i.item->>'badges_shown')::bool  AS treated
    FROM events e,
         LATERAL jsonb_array_elements(e.payload->'listing_ids') AS i(item)
    WHERE e.event_type = 'feed_view'
""", engine)

contacts = pd.read_sql(
    "SELECT session_id, listing_id FROM events WHERE event_type='contact_click'",
    engine)

# did this exact impression turn into a contact in the same session?
imp["contacted"] = imp.merge(contacts, how="left", indicator=True,
                             on=["session_id", "listing_id"])["_merge"].eq("both")

g = imp.groupby("treated")["contacted"].agg(["sum", "count"])
z, p = proportions_ztest(g["sum"], g["count"])
```

```
print(g.assign(rate=g["sum"] / g["count"]))
print(f"lift p-value: {p:.4f}")
print(proportion_confint(g["sum"], g["count"], method="wilson"))
```

PRICES, SPEED OF SALE, AND THE SEASON

```
items = pd.read_sql("""
    SELECT l.*, u.college, u.nationality, u.location, u.grade
    FROM listings l JOIN users u ON u.id = l.seller_id
""", engine)
items["price"] = items.price_cents / 100

# what things go for, and whether "unused" carries a premium
print(items.groupby(["category", "condition"])["price"]
           .agg(["count", "median", "mean"]).round(2))

# how long a sold item sat in the feed
sold = items[items.status.eq("sold")].copy()
sold["days_listed"] = (sold.sold_at - sold.created_at).dt.days
print(sold.groupby("category").days_listed.median().sort_values())

# posts per active seller per week — growth does not fake a season
weekly = (items.set_index("created_at")
               .resample("W")
               .agg(posts=("id", "size"), sellers=("seller_id", "nunique")))
weekly["posts_per_seller"] = weekly.posts / weekly.sellers
weekly.posts_per_seller.plot(title="Listings per active seller, by week")
```

DECIDE THE DEFINITIONS BEFORE LAUNCH, NOT AFTER

Write down what counts as a "successful contact" and what circle depth means *before* the first real user signs in. Choosing them after you have seen the numbers is the fastest way to turn a real finding into an unpublishable one.

Build order, step by step

Ten steps in dependency order — each one is finishable in a sitting or two, and each ends with something you can demo. Four weeks with four people if you split by step group: one on sign-in and profile, one on the feed, one on posting and photos, one on events and the notebook.

01 SETUP · HALF A DAY

Accounts and the repo skeleton

Create two Supabase projects (dev and prod), a Vercel project, a Render service, and a Resend account. Buy the domain and set up SPF and DKIM records — if you skip this, the sign-in emails land in spam and nothing else you build matters.

One repo, three folders: `/web` for Next.js, `/api` for FastAPI, `/analytics` for notebooks.

02 BACKEND · 1 DAY

Database schema and migrations

Write the SQLAlchemy models for all four tables from section 2, then `alembic revision --autogenerate` and review the migration by hand before applying it. Build the `events` table now, not later — retrofitting instrumentation means throwing away the first weeks of data.

03 SIGN-IN · 2 DAYS

Magic-link sign-in, gated to Columbia

This is the whole authentication system. There are no passwords anywhere in it.

1. The student types `uni1234@columbia.edu` into one field and presses *Send me a link*.
2. Next.js posts it to `POST /v1/auth/magic-link`. FastAPI checks the domain against an allowlist held in config, not code — `columbia.edu` and `gsb.columbia.edu` at launch. A Gmail address is rejected here with a plain explanation, before any email is sent.
3. FastAPI calls Supabase `signInWithOtp()`, which emails a single-use link through Resend. The link expires 15 minutes after it is sent and dies the moment it is used.
4. **Enforce the allowlist a second time in Supabase's `before-user-created` auth hook.** The check in step 2 is only for the error message; this one is the actual gate. Nothing that reaches the database can create a non-Columbia account, whatever the app layer does.
5. The student clicks the link, which opens `/auth/callback` in Next.js. That route exchanges the code for a Supabase session and stores it in an httpOnly cookie.
6. Every later call to FastAPI carries the session's JWT. FastAPI verifies it against Supabase's signing keys on every protected request — one shared dependency, one place to get it right.
7. If the token is valid but no `users` row exists, the API answers `profile_required` and Next.js sends them to onboarding.

Demo gate: a teammate signs up with their Columbia address and gets in; a Gmail address is turned away with a message that explains why.

04

SIGN-IN · 1 DAY

Onboarding and profile

A first-run form that collects all seven fields. The four matching fields are dropdowns — never a text box, or `=` stops working and every filter in the product silently degrades. Prefill `college` from the email subdomain where it is unambiguous (`@gsb.columbia.edu` → CBS) and leave it editable. No browsing until the profile is complete.

Build the edit page and the delete-my-account flow in the same sitting — deletion clears the personal fields, delists the person's items, and keeps their event rows under an opaque ID.

05

BACKEND · 2 DAYS

The feed endpoint

`GET /v1/feed` runs the section 3 query with the viewer's own attributes bound as parameters. Return badges, the shared-only seller fields, and `total_count`. Do not fetch full profiles and hide fields in the frontend — the projection *is* the privacy rule.

Assign the badge-visibility coin flip here, per impression, and record it in the `feed_view` event so the experiment in section 4 has data from day one.

06

BACKEND · 2 DAYS

Posting, photos, and contact

`POST /v1/photos` takes the upload, checks the type and size, resizes to 1600px, re-encodes to WebP, strips the metadata (phone photos carry GPS coordinates), and writes to Storage. The browser never writes to Storage directly, so the strip cannot be bypassed.

`POST /v1/listings` validates with Pydantic and refuses to publish without at least one photo. `PATCH` handles edit, mark sold and delist, owner only.

`POST /v1/listings/{id}/contact` returns the seller's email and phone and logs the event in the same call — *the reveal is the measurement*. Rate-limit it. Do not build messaging.

07

FRONTEND · 4 DAYS

The five screens

There are only five, and they are the entire product.

SIGN IN

One email field, one button, one "check your inbox" state.

ONBOARDING

Seven fields, four of them dropdowns.

FEED

Grid of item cards, four filter toggles, a live count, category chips.

ITEM DETAIL

Photos, price, condition, description, match badges, one Contact button.

POST AN ITEM

Six fields plus a photo picker, with a preview.

Keep filter state in the URL (`?college=1&country=1&category=furniture`) so pages are shareable and the back button behaves. Server-render the feed from the API response.

Default every filter to off, and never remember a narrow filter across sessions. When a filter empties the feed, name the one whose removal brings back the most items — a first view with three results is how you lose a user permanently.

08

WIRING · 1 DAY

Connecting the parts

Four connections, and each one has a specific failure mode worth testing.

LINK	HOW	TEST IT BY
Next.js → FastAPI	`NEXT_PUBLIC_API_URL` ; attach the Supabase JWT as a Bearer header on every call	Calling a protected endpoint with no token and getting 401
FastAPI → Supabase Auth	Verify the JWT against Supabase's signing keys in one shared dependency	Sending a hand-edited token and getting 401
FastAPI → Postgres	`DATABASE_URL` , SQLAlchemy pool. Supabase row-level security denies browser roles outright	Trying to read `listings` from browser JS and being refused
Browser → events	`navigator.sendBeacon` to `POST /v1/events` so navigating away doesn't drop the event	Toggling a filter, navigating instantly, checking the row landed

Set CORS on FastAPI to the Vercel domains only. Keep two environments end to end: dev Supabase for previews, prod Supabase only for `main` . Alembic runs as a pre-deploy step on Render.

09

ANALYTICS · 2 DAYS

Events and the notebook

Emit all eight event types from section 2 with a session ID generated once per browser session. Create the read-only database role, create the views, and run the section 4 notebook against test browsing — not on launch day, when the first real data arrives.

Demo gate: the badge experiment table shows real treated and control rows from your own test session.

10

LAUNCH · 3 DAYS

Harden, seed, open

- Rate-limit sign-in link requests, contact reveals, and posting.
- **Write the one test that must exist:** an automated check that no API response to student B ever contains an attribute of student A that B does not share.
- Keyboard navigation, visible focus, contrast, form labels.
- Privacy and terms pages live; a consent checkbox at sign-up covering research use of behavioral data.
- Confirm backups actually restore. A weekly `pg_dump` artifact is the belt to Supabase's suspenders.
- Recruit sellers by hand and get **20 real items posted before you announce anything.** An empty marketplace gets exactly one visit per student.

LionsList · internal-only edition · v2.0
Simplified from the September 2026 pilot spec: external listings, the eBay integration and the industry attribute removed; profile reduced to seven fields and items to six.
Columbia University.